数值分析——线性方程组上机报告

未央-水木 22 郁然

1 实验目的

本次上机作业需要用线性方程的直接解法和迭代解法处理 Hilbert 矩阵所构成的线性方程组的求解问题。用到的方法有：Gauss 消去法，Cholesky 分解法，正则化方法，共轭梯度法和 GMRES 方法。Hilbert 矩阵 $H_n \in \mathbb{R}^{n\times n}$，n=10，11,…,15。取 $x=(1,\ldots,1)^{\mathrm{T}} \in \mathbb{R}^n$，令 $b_n = H_n x$，求解方程 $H_n y = b_n$，观察可知精确解 $y=(1,\ldots,1)^{\mathrm{T}}$。

我们期望通过这次作业，比较几种方法直接效果与误差，并期望求出尽可能精确的解。

2 实验方法详述

2.1 Gauss 消元法

首先采用 Gauss 消元法进行处理，代码见附录 2，采用方法为顺序 Gauss 消元法。主要利用的公式为：

$$l_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}, i = k+1,\cdots,n.$$

$$a_{ij}^{(k+1)} = \begin{cases} a_{ij}^{(k)}.i = 1,2,\cdots,k.j = 1,2,\cdots,n. \\ 0.i = k+1,\cdots,n.j = 1,2,\cdots,k. \\ a_{ij}^{(k)} - l_{ik}a_{kj}^{(k)}.i,j = i = k+1,\cdots,n. \end{cases}$$

$$b_i^{(k+1)} = \begin{cases} b_i^{(k)}.i = 1,2,\cdots,k. \\ b_i^{(k)} - l_{ik}b_k^{(k)}.i = k+1,\cdots,n. \end{cases}$$

2.2 Cholesky 分解法

采用 Cholesky 分解法，首先 Hilbert 矩阵 H_n 对称正定，故存在唯一的对角元素为正数的下三角矩阵 L，使

$$H_n = LL^{\mathrm{T}}$$

利用 $Lz=b$ 和 $L^{\mathrm{T}}y=z$ 求解方程组即可得到结果，代码见附录 3.

2.3 正则化方法

采用吉洪诺夫正则化方法对上述结果进行改进，代码见附录 4.在运算过程中，为计算简便考虑，直接选取 $\alpha=0.02078$，在 $n=10$ 时，与精确解较为接近。

2.4 共轭梯度法

共轭梯度法在寻优过程中首先在 $x^{(k)}$ 处寻找搜索方向 $p^{(k)}$ 的最优：

$$x^{(k+1)}=x^{(k)}+\alpha_k p^{(k)}$$

$$\alpha_k=\frac{(r^{(k)},r^{(k)})}{(p^{(k)},Ap^{(k)})}$$

计算残差：

$$r^{(k+1)}=r^{(k)}-\alpha_k Ap^{(k)}$$

利用当前 $x^{(k+1)}$ 处的残差 $r^{(k+1)}$ 和前一迭代点处的搜索方向 $p^{(k)}$ 进行修正：

$$p^{(k+1)}=r^{(k+1)}+\beta_k p^{(k)}$$

$$\beta_k=\frac{(r^{(k+1)},r^{(k+1)})}{(r^{(k)},r^{(k)})}$$

重复上述过程直至误差在符合范围内，代码见附录 5。

2.5 GMRES 法

GMRES 算法过程：

1、令 m = 1

2、定义 Krylov 子空间 K_m

3、找出 x_0+K_m 中的最佳近似解

4、判断近似解是否满足精度要求，如果满足则返回结果，否则递增 m，返回步骤 2。

算法代码见附录 6。

3 实验结果讨论

3.1 Gauss 消元法

当 n=10 时，解得 y 如下图所示（为显示方便，下图实际上为 y^T，下同理）

1	1	0.999998	1.00002	0.999903	1.000267	0.999563	1.000421	0.999779	1.000048

采用向量的无穷范数估计误差，得到

$$\|x-y\|_\infty = 0.000436911529730021$$

当 n=11 时，y^T 为

1	1.000001	0.999986	1.000158	0.999064	1.003286	0.992856	1.009726	0.99193	1.00373	0.999264

误差为

$$\|x-y\|_\infty = 0.009726486881558$$

当 n=12 时，y^T 为

1	1.000003	0.999916	1.001125	0.991859	1.035385	0.902315	1.175419	0.795768	1.148663	0.938525	1.011022

误差为

$$\|x-y\|_\infty = 0.204232243715017$$

当 n=13 时，y^T 为

1	0.999984	1.000614	0.989763	1.091975	0.501003	2.740852	-3.03606	7.283884	-5.49353	5.270874	-0.61819	1.268829

误差为

$$\|x-y\|_\infty = 6.493529661418352$$

当 n=14 时，y^T 为

1.00E+00	0.99999	1.00031	0.99622	1.019948	0.981738	0.675197	2.922896	-4.4981	10.51596	-9.42821	8.094763	-1.74096	1.460245

误差为

$$\|x-y\|_\infty = 10.428207690148314$$

当 n=15 时，y^T 为

1	1.000002	0.999995	0.99864	1.025668	0.781933	2.066841	-2.27904	7.532393	-7.35505	7.380667	-1.12904	0.425749	1.733284	0.817952

误差为

$$\|x-y\|_\infty = 8.355047567109128$$

总体上来看，Gauss 消元法得到的解的误差随着 n 的增大而增大，甚至出现了负数的情况。在 n 取 10 和 11 时，解与精确值的误差相对较小，而当 $n \geq 12$ 时，

此时的误差较大。

3.2 Cholesky 分解法

当 n=10 时，解得 y 如下图所示（为显示方便，下图实际上为 y^T，下同理）

2.928968	24.2385281	288.5779221	3771.041181	51518.8986	722920.154	10330243	1.5E+08	2.19E+09	3.23E+10

采用向量的无穷范数估计误差，得到

$$\|x-y\|_\infty = 32279160641$$

当 n=11 时，y^T 为

3.019877	25.2385281	302.4240759	3971.041181	54458.8986	766579.154	10983131	1.59E+08	2.34E+09	3.45E+10	5.12E+11

误差为

$$\|x-y\|_\infty = 512156520451$$

当 n=12 时，y^T 为

3.103211	26.1616051	315.2812188	4157.707848	57215.1486	807669.977	11599747	1.69E+08	2.48E+09	3.67E+10	5.45E+11	7.73E+12

误差为

$$\|x-y\|_\infty = 7728749576209$$

当 n=13 时，y^T 为

3.180134	27.0187479	327.2812188	4332.707848	59809.26625	846477.977	12183909	1.78E+08	2.61E+09	3.87E+10	5.76E+11	8.18E+12	6.44E+13

误差为

$$\|x-y\|_\infty = 64358028062259$$

当 n=14 时，y^T 为

3.251562	27.8187479	338.5312188	4497.41373	62259.26625	883243.451	12738863	1.86E+08	2.74E+09	4.07E+10	6.06E+11	8.61E+12	6.78E+13	1.34E+15

误差为

$$\|x-y\|_\infty = 1340250270876391$$

事实上，由于取舍误差，此时的 L 中已经有元素为复数。即在计算过程中计算机系统已经认为这不再是一个正定的矩阵。

当 n=15 时，y^T 为

3.318229	28.5687479	349.1194541	4652.969286	64580.31888	918170.651	13267391	1.94E+08	2.86E+09	4.25E+10	6.34E+11	9.03E+12	7.12E+13	1.41E+15	2.23E+13

误差为

$$\|x-y\|_\infty = 1407541896803500$$

关于这个算法，笔者反复检查了程序代码未能检查出原理上的错误，然而从10 开始，解的误差已经相当可观，因此笔者只能将误差归结到计算时候的取舍误差造成的数据偏差。事实上，如果在 matlab 中采用 inv 函数运算 L 的逆矩阵，可以得到误差相对较小的结果，但与书上所提供的算法有细微差别，故未采纳。

3.3 正则化方法

当 n=10 时，解得 y 如下图所示（为显示方便，下图实际上为 y^T，下同理）

0.999596	1.20651144	1.145637421	1.05591178	0.970267799	0.89429621	0.828016	0.770248	0.719691	0.675187

采用向量的无穷范数估计误差，得到

$$\|x-y\|_\infty = 0.324813338987688$$

当 n=11 时，y^T 为

0.980384	1.2150727	1.16790818	1.085348724	1.003524821	0.92951806	0.864116	0.806568	0.755823	0.710875	0.670854

误差为

$$\|x-y\|_\infty = 0.329145959129633$$

当 n=12 时，y^T 为

0.962946	1.22036703	1.185911315	1.11043397	1.032629989	0.96087616	0.896658	0.839623	0.788961	0.743817	0.703418	0.667096

误差为

$$\|x-y\|_\infty = 0.332903554861360$$

当 n=13 时，y^T 为

0.947141	1.22305741	1.200383891	1.131852803	1.058193086	0.98890639	0.92611	0.869825	0.819469	0.774336	0.733747	0.6971	0.663878

误差为

$$\|x-y\|_\infty = 0.336121772649122$$

当 n=14 时，y^T 为

0.932837	1.22365156	1.211900415	1.15014395	1.080695854	1.01403324	0.952846	0.897502	0.847638	0.802688	0.762069	0.725243	0.691739	0.661146

误差为

$$\|x-y\|_\infty = 0.338853938541462$$

当 n=15 时，y^T 为

0.919912	1.22254771	1.220920027	1.165742014	1.100529231	1.03660248	0.97717	0.922922	0.873703	0.829082	0.788568	0.75169	0.718019	0.687179	0.658842

误差为

$$\|x-y\|_\infty = 0.341158125385461$$

从上述过程中可以看到，正则化方法虽然起始的误差相较于 Gauss 消元法较大，但是随着 n 的增加误差的增加并不迅速，算法具有相对较好的稳定性。而另一方面，起始的误差可以通过α值的改变进行进一步的优化，此处的α只是取了一个误差较小的值，如果进一步计算优化应该可以得到更小的误差。

3.4 共轭梯度法

直接令程序运行至多 n 步，即调整 norm(r1)==0，寻找尽可能精确的解。

当n=10 时，解得 y 如下图所示（为显示方便，下图实际上为 y^T，下同理）

0.999999	1.00004219	0.999719235	1.0005988	0.999825752	0.9994953	0.999891	1.000432	1.000466	0.99953

采用向量的无穷范数估计误差，得到

$$\|x-y\|_\infty = 0.000598800013283495$$

当n=11 时，y^T为

1	0.9999968	1.000027728	0.99990242	1.000118847	1.00003102	0.999902	0.999926	1.000059	1.000124	0.999912

误差为

$$\|x-y\|_\infty = 0.000259246533110893$$

当n=12 时，y^T为

1	0.99999429	1.000050589	0.999842301	1.000145782	1.00009432	0.99991	0.999855	0.999969	1.000128	1.000151	0.999859

误差为

$$\|x-y\|_\infty = 0.000157698596743638$$

当n=13 时，y^T为

1	0.99999021	1.00008085	0.999772653	1.000160558	1.00016709	0.999947	0.99981	0.999866	1.000047	1.000202	1.000165	0.999791

误差为

$$\|x-y\|_\infty = 0.000227346546537221$$

当n=14 时，y^T为

1	0.99998453	1.000119765	0.999693583	1.000160779	1.00024369	1.000011	0.999797	0.999774	0.999927	1.000143	1.00027	1.000165	0.999711

误差为

$$\|x-y\|_\infty = 0.000306417113960622$$

当n=15 时，y^T为

1	0.99998087	1.000136196	0.999693484	1.000095644	1.0002524	1.000091	0.999867	0.999765	0.99983	1.000006	1.00019	1.000267	1.00013	0.999694

误差为

$$\|x-y\|_\infty = 0.000306515509627125$$

可以看到，用共轭梯度法处理问题时误差较小，克服了最速下降法下降速度慢的问题，可以看作是一种较为普遍且有效的计算方法。

3.5 GMRES 法

取残差容差为1×10^{-10}，对不满足要求的进行迭代操作。

当n=10 时，解得 y 如下图所示（为显示方便，下图实际上为y^T，下同理）

0.999999	1.00004222	0.99971912	1.000599171	0.999825537	0.99949498	0.999891	1.000432	1.000466	0.99953

采用向量的无穷范数估计误差，得到

$$\|x-y\|_\infty = 0.000599171382986263$$

gmres 在解的 迭代 6 处收敛。

当n=11 时，y^T为

0.999997	1.00007412	0.999549815	1.000820895	0.999974676	0.99934871	0.999565	1.000216	1.000687	1.00048	0.999284

误差为

$$\|x-y\|_\infty = 0.000820895389500586$$

gmres 在解的 迭代 6 处收敛。

当n=12 时，y^T为

1	0.99999428	1.000050639	0.999842292	1.000145728	1.00009428	0.99991	0.999855	0.999969	1.000128	1.000151	0.999859

误差为

$$\|x-y\|_\infty = 0.000157708114714605$$

gmres 在解的 迭代 7 处收敛。

当n=13 时，y^T为

1	0.99999021	1.000080845	0.999772698	1.000160545	1.00016704	0.999947	0.99981	0.999866	1.000047	1.000202	1.000165	0.999791

误差为

$$\|x-y\|_\infty = 0.000227301800643231$$

gmres 在解的 迭代 7 处收敛。

当n=14 时，y^T为

1	0.99998456	1.000119705	0.999693601	1.000160879	1.00024369	1.00001	0.999797	0.999774	0.999927	1.000143	1.00027	1.000165	0.999711

误差为

$$\|x-y\|_\infty = 0.000306399165541315$$

gmres 在解的 迭代 7 处收敛。

当n=15 时，y^T为

1.000001	0.99997713	1.000167485	0.999607066	1.000145552	1.00031862	1.000094	0.999815	0.999705	0.999802	1.000025	1.000246	1.000327	1.000149	0.999619

误差为

$$\|x-y\|_\infty = 0.000392934159431979$$

gmres 在解的 迭代 7 处收敛。

运用 GMRES 法时，如果采用较多次数的迭代，可以得到误差较小的解，但是迭代计算过程中可能有较大的计算量，时间上耗费较大。

4 实验总结

通过这次实验可以看出，Gauss 和 Cholesky 方法在处理较为病态的方程组时容易受到取舍误差的影响，相较之下采用正则化的方法可以较为有效的增加算法的稳定性。而迭代法求解的过程中大大减小了解的误差，相比直接法处理病态方程组显得更为有效、普遍。

对 Hilbert 阵的条件数进行简单计算，$n=10$ 时条件数已经是10^{13}量级的数，$n=15$时条件数更是达到了10^{17}量级。随着n的增加，Hilbert 阵的条件数指数增大，所以对于直接法其误差必然是相对较大的。

在处理病态方程时，迭代法虽然可以得到较为精确的解，但是其计算量相对较大，尤其是多次迭代时需要耗费较多时间，因此，选择合适的初值与适当的误差范围也是有必要的，如果条件允许，也可以借助计算机等工具。

附录 程序代码

注：本次上机实验中，利用 matlab 完成相关操作，以下代码均为 matlab 中的代码。

1 Hilbert 矩阵的生成（以 n=10 为例）

```
n = 10;
A  =zeros(n,n);
for i = 1:n
    for j = 1:n
        A(i,j) = 1/(i+j-1);
    end
end
format rat % 用分数进行显示 A
x([1:n],1)=1
b=A*x
```

2 Gauss 消元法

```
n = length(b);
for k = 2:n
    for i = k:n
        l = A(i, k-1) / A(k-1, k-1);
        for j = 1:n
            A(i, j) = A(i, j) - l * A(k-1, j);
        end
        b(i) = b(i) - l * b(k-1);
    end
end
% Perform back substitution
% preallocate memory for y
y = nan(n,1);
y(n) = b(n) / A(n,n);
for k = n-1 : -1 : 1
    m = 0;
    for j = k+1 : n
        m = m + A(k, j) * y(j);
    end
    y(k) = (b(k) - m) / A(k,k);
end
```

3 Cholesky 分解法

```
n=size(A,1);
L=zeros(n);
```

```
L(1,1:n)=A(1,1:n);
L(2:n,2:n)=A(2:n,2:n);
for i=1:1:n
    L(i,i)=sqrt(L(i,i));
    u=zeros(1,n-i);
    for j=i+1:1:n
        L(i,j)=L(i,j)/L(i,i);
        u(1,j-i)=L(i,j);
    end
U=u'*u;
L(i+1:n,i+1:n)=L(i+1:n,i+1:n)-U;
end
L=triu(L);
z = zeros(n,1);
y = zeros(n,1);
for j=1:n
    z(j) = (b(j)-L(j,1:j-1)*z(1:j-1))/L(j,j);
end
L = L';
for k=n:-1:1
    y(k)=(z(k)-L(k,k+1:n)*z(k+1:n))/L(k,k);
end
```

4 正则化方法

```
bn=A'*b
y=inv(0.02078*eye(n)+A'*A)*bn
```

5 共轭梯度法

```
n=length(b);
y=zeros(n,1);
r=b-A*y;
d=r;
i=1;
for k=1:n
    alpha=(r'*d)/(d'*A*d);
    y=y+alpha*d;
    r1=b-A*y;
    error(i)=norm(r1);
    bt=-(r1'*A*d)/(d'*A*d);
    d=r1+bt*d;
    r=r1;
    i=i+1;
    if norm(r1)==0
        break;
```

```
    end
end
```

6 GMRES 法

```
y=gmres(A,b,[],1e-10)
```